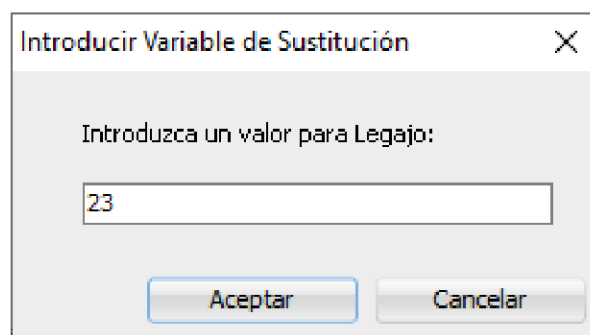
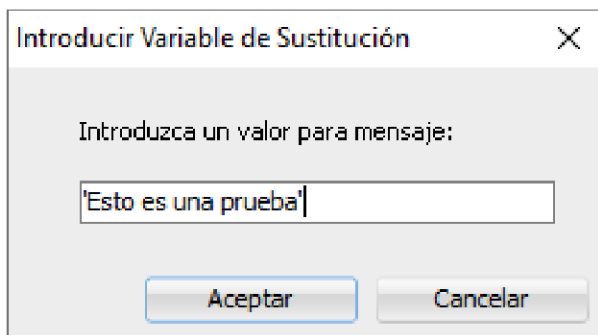


Lenguaje PL/SQL Oracle – Parte 7

Ingresar un Dato por Teclado

- a. **Variables de Sustitución.** En PL/SQL para tomar datos desde el teclado se usa las variables de sustitución. En el código estas variables van precedidas del **&**. O sea, si dentro de la ejecución de un bloque necesitamos solicitarle al usuario un dato lo haremos colocando el símbolo **&** seguido de un identificador que será mostrado en una ventana de diálogo en la que se nos pide el valor deseado. Por ejemplo, **&mensaje** o **&Legajo**.



La variable ingresada debe ser del mismo tipo que la declarada. Si es un **VARCHAR** se coloca entre comillas simple el contenido de la variable, si es número no es necesario. Si el dato que se ingresará es una cadena de caracteres y no se lo ingresa entre apóstrofes, comillas simples, dará un error en tiempo de ejecución. El nombre del identificador se muestra como parte del mensaje, ya sea en minúscula, mayúscula o como se lo haya escrito.

Las variables de sustitución pueden utilizarse una única vez y pierden su contenido al finalizar la ejecución del bloque donde se encuentre, si la volvemos a utilizar pedirá nuevamente que se la ingrese.

```
DECLARE
    nombre VARCHAR2(25);
BEGIN
    nombre := '&nombre_usuario'; -- Variable de sustitución
    DBMS_OUTPUT.PUT_LINE('Hola, ' || nombre || '!');
END;
/
```

En la zona de Salida de Script muestra las declaraciones originales de las variables y luego las declaraciones con los contenidos de los identificadores.

- b. **Variables de Sustitución Permanentes.** Las variables de sustitución permanentes ayudan a realizar tareas con valores ingresados por el usuario desde el teclado, más de una vez, pero habiéndoles ingresado una única vez su contenido. Para usar las variables de sustitución en forma permanente, simplemente en lugar de utilizar solo un **&** antes del nombre de la variable, la primera vez se utiliza **&&** y las veces subsiguientes, cuando se desea usar su contenido se utiliza **&**. El valor de la variable una vez ingresado tiene vigencia en la sesión, por lo tanto no volverá a pedir el valor para esa variable en la sesión.

```
SELECT * FROM criminales WHERE edad < &&numero;
```

Puede también utilizarse en un bloque PL/SQL

```
DECLARE
    nombre VARCHAR2(25);
BEGIN
    nombre := '&&usuario'; -- Variable de sustitución
    DBMS_OUTPUT.PUT_LINE('Hola, ' || nombre || '!');
END;
```

Nota: Si el contenido es de caracteres es conveniente que el nombre de la variable de sustitución tanto las comunes como las permanentes esté encerrado entre las comillas simples para no tener que darlas con el contenido.

- c. **Variables de Enlace.** Otra forma de ingresar datos es a través del uso de variables de enlace conocidas por variables bind. En PL/SQL, las variables de enlace, son variables que se utilizan dentro de una sentencia SQL, en lugar de sustituir directamente los valores en la consulta. Esto tiene ventajas en términos de seguridad, rendimiento y reutilización de planes de ejecución.

- *Seguridad:* Las variables bind ayudan a prevenir ataques de inyección SQL. La base de datos interpreta los valores de las variables bind como datos y no como código, evitando la ejecución de código malicioso.
- *Rendimiento:* Las bases de datos pueden reutilizar el plan de ejecución de una sentencia SQL si es la misma, incluso si cambia el valor de las variables bind. Esto ahorra tiempo de procesamiento y mejora la eficiencia.
- *Reutilización:* Las variables bind permiten reutilizar el mismo código SQL con diferentes valores, sin necesidad de recompilar la sentencia cada vez.

Para utilizar las variables bind en PL/SQL se debe:

1. Declararlas: Las variables de enlace se declaran fuera del bloque anónimo mediante la sentencia VARIABLE o VAR. Las variables de caracteres se le puede asignar un tamaño o no como se lo desee, pero a las variables numéricas no se le puede dar tamaño

```
VARIABLE b_nombre VARCHAR2(30);
VARIABLE b_distrito VARCHAR2;
VARIABLE b_dire VARCHAR2(50);
VAR b_edad NUMBER(5);
```

2. Asignarles contenido: Puedes asignar valores a las variables de enlace dentro del bloque PL/SQL, como cualquier variable normal. Colocándole antes del nombre de la variable : o utilizando la orden EXEC fuera del bloque anónimo.

```
EXEC :b_dire:='Av. Rivadavia 12345';
DECLARE
    v_nombre VARCHAR2(20);
BEGIN
    :b_distrito:='Comuna 14';
.....
```

3. Utilizarlas en sentencias SQL: En la sentencia SQL, puedes usar las variables de enlace con **:variable_name** para sentencias con la sintaxis de PL/SQL.

```
SELECT nombres INTO :b_nombre FROM clientes WHERE cod_cliente='C0003';
```

4. Ejecución: La base de datos procesará la sentencia SQL, reemplazando los marcadores de posición por los valores de las variables bind. Se debe ejecutar el bloque anónimo como SCRIPT incluyendo en la selección al bloque anónimo, las definiciones de las variables de enlace y las asignaciones de contenido de las variables de enlace mediante EXEC.

Trabajando con Colecciones en PL/SQL

PL/SQL soporta para registros, varios tipos de datos compuestos, le permite escribir código que es simple, limpio y fácil de mantener. En lugar de trabajar con largas listas de variables o parámetros, puede trabajar con un registro que contenga toda esa información. Los registros definidos por el usuario ofrecen la flexibilidad de construir su propio tipo de datos compuestos, reflejando los requisitos específicos del programa que pueden no estar representados por una tabla relacional.

Un registro PL/SQL, es un tipo de datos compuesto por uno o más campos, Otro tipo de datos compuestos son las colecciones. Una colección Oracle PL/SQL es una matriz unidimensional; consta de uno o más elementos accesibles a través de un valor de índice. Las colecciones se utilizan en algunas de las funciones de optimización de rendimiento más importantes de PL/SQL, como:

- **BULK COLLECT.** Son sentencias SELECT que recuperan varias filas con una sola búsqueda, lo que aumenta la velocidad de recuperación de datos.
- **FORALL.** Inserciones, actualizaciones y eliminaciones que usan colecciones para cambiar varias filas de datos muy rápidamente.
- **Funciones de la tabla.** Funciones PL/SQL que devuelven colecciones y se pueden llamar en la cláusula FROM de una declaración SELECT.

También puede usar colecciones para trabajar con listas de datos en su programa que no están almacenados en las tablas de la base de datos. Comencemos definiendo un vocabulario común de colecciones:

- **Valor de índice (Index Value).** La ubicación de los datos en una colección. Los valores de índice suelen ser números enteros, pero un tipo de colección también puede ser una cadena.
- **Elemento (Element).** Los datos almacenados en un valor de índice específico en una colección. Los elementos de una colección son siempre del mismo tipo (todos ellos son cadenas, fechas o registros). Las colecciones de PL/SQL son homogéneas.
- **Escaso (Sparse).** Una colección es escasa si hay al menos un valor de índice entre los valores de índice definidos más bajo y más alto que no está definido. Por ejemplo, una colección dispersa tiene un elemento asignado al valor de índice 1 y otro al valor de índice 10, pero nada intermedio. Lo opuesto a una colección escasa es una densa.
- **Método (Method).** Un método de colección es un procedimiento o función que proporciona información sobre la colección o cambia el contenido de la colección. Los métodos se adjuntan a la variable de colección con notación de puntos (sintaxis orientada a objetos), como en `mi_coleccion.FIRST`.

Tres tipos de colecciones

Las colecciones se han mejorado de varias maneras a lo largo de los años y en todas las versiones de Oracle Database. Ahora hay tres tipos de colecciones para elegir, cada una con su propio conjunto de características y cada una se adapta mejor a una circunstancia diferente:

- **Arreglo de Tamaño Variable (Varray).** El varray (arreglo de tamaño variable) se puede utilizar en bloques PL/SQL, en sentencias SQL y como tipo de datos de columnas en tablas. Los Varrays siempre son densos e indexados por enteros. Cuando se define un tipo de varray, se debe especificar el número máximo de elementos permitidos en una colección declarada con ese tipo.
- **Tablas Anidada.** La tabla anidada se puede utilizar en bloques PL/SQL, en sentencias SQL y como tipo de datos de columnas en tablas. Las tablas anidadas pueden ser escasas, pero casi siempre

son densas. Solo se pueden indexar por número entero. Puede utilizar el operador MULTISSET para realizar operaciones de conjuntos y realizar comparaciones de igualdad en tablas anidadas.

- **Matriz Asociativa.** Es el primer tipo de colección disponible en PL/SQL, originalmente se las llamaba "tabla PL/SQL" y sólo se puede usar en bloques PL/SQL. Las matrices asociativas pueden ser escasas o densas y pueden indexarse por enteros o cadenas de caracteres.

La matriz asociativa es el tipo de colección que se usa con más frecuencia, pero las tablas anidadas tienen algunas funciones únicas y poderosas (como los operadores MULTISSET) que pueden simplificar el código necesario para usar su colección. No son muy utilizados los Varray, ya que es muy difícil que de antemano se sepa cuál es el número máximo de elementos que se definirá en la colección. Mediante ejemplos veremos algunos aspectos de las colecciones:

Ejemplo de Varrays

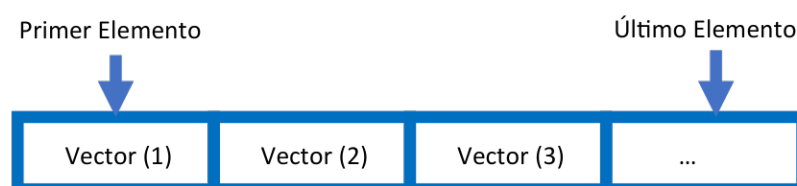
```
1 CREATE OR REPLACE PROCEDURE orden_alfa (palabra IN VARCHAR2)
2 IS
3   palabra_aux VARCHAR2(50);
4   TYPE vector IS VARRAY(50) OF VARCHAR2(1);
5   letra vector;
6   cantidad NUMBER(2);
7   palabra_ordenada VARCHAR2(50);
8   aux VARCHAR2(1);
9 BEGIN
10  palabra_aux:=UPPER(palabra);
11  letra:=vector(substr(palabra_aux,1,1));
12  cantidad:=LENGTH(palabra);
13  FOR I IN 2..cantidad LOOP
14    letra.EXTEND;
15    letra(letra.LAST):=substr(palabra_aux,I,1);
16  END LOOP;
17  FOR I IN 1..cantidad-1 LOOP
18    FOR J IN I+1..cantidad LOOP
19      IF letra(I)>letra(J) THEN
20        aux:=letra(J);
21        letra(J):=letra(I);
22        letra(I):=aux;
23      END IF;
24    END LOOP;
25  END LOOP;
26  FOR I IN 1.. letra.COUNT LOOP
27    palabra_ordenada:=palabra_ordenada || letra(I);
28  END LOOP;
29  dbms_output.put_line(palabra_ordenada);
30 END;
```

La siguiente tabla proporciona una explicación del código:

Líneas	Explicación
4	Se declara un nuevo tipo de varray. Cada elemento de una colección declarada con este tipo será una cadena de caracteres cuya longitud máxima es 1 caracter.
5	Se declara un varray (letra) en función del nuevo tipo de colección. En esta misma instrucción se puede inicializar el vector a la función constructora. Tener en cuenta que también se asigna un valor predeterminado a cada variable llamando a una función constructora que tiene el mismo nombre que el tipo y se crea junto con el objeto.
11	Se inicializa el vector mediante la función constructora y a su vez dándole el valor del primer elemento.
14	Se hace espacio en la colección de letra para el resto de sus elementos llamando al método EXTEND (en este caso hemos utilizado un ciclo de repetición para tomar cada uno de los elementos del vector)
15	Se asigna a cada elemento del vector una letra de la palabra ingresada en la llamada del procedimiento. Tener en cuenta el uso de la sintaxis típica de arreglos de una sola dimensión para identificar un elemento en el arreglo: nombre_arreglo (valor_índice) En este ejemplo se utilizó el método LAST para tomar en cada paso del ciclo de repetición el valor del índice indicado. Podríamos haber utilizado el método COUNT que nos daría la cantidad de elementos usados del vector. Recordemos que este tipo de colección es densa, continua.
19 – 22	Utilizamos cada uno de los valores de cada variable de la colección mediante el uso de los índices en los ciclos de repetición. En esta parte del procedimiento se ordenan los elementos del vector. Se puede observar que los elementos de la colección pueden cambiar su contenido cuantas veces sea necesario.
27 - 28	Recorremos cada uno de los elementos del vector en forma ordenada, desde el primero hasta el último. Así que se recorrerán todos los elementos de la colección, desde 1 hasta COUNT (la cantidad de elementos definidos en la colección) y utiliza el elemento encontrado en cada valor de índice a fin de concatenarlos en una nueva palabra.

PL/SQL proporciona esta estructura de datos llamada VARRAY, que puede almacenar una colección secuencial de tamaño fijo de elementos del mismo tipo. Un varray se utiliza para almacenar una colección ordenada de datos; sin embargo, suele ser más conveniente considerar un array como una colección de variables del mismo tipo.

Todas las varrays constan de ubicaciones de memoria contiguas. La dirección más baja corresponde al primer elemento y la más alta al último.



En el entorno Oracle, el índice inicial de los varrays siempre es 1. Puede inicializar los elementos de un varray mediante el método constructor del tipo varray, que tiene el mismo nombre que el varray. Los varrays son matrices unidimensionales. Un varray está automáticamente vacío al declararse y debe inicializarse para poder referenciar sus elementos.

Ejemplo de Tablas Anidadas

```
1 DECLARE
2   TYPE lista_de_nombres_t IS TABLE OF VARCHAR2 (100);
3
4   familiafeliz lista_de_nombres_t := lista_de_nombres_t ();
5   hijos      lista_de_nombres_t := lista_de_nombres_t ();
6   padres     lista_de_nombres_t := lista_de_nombres_t ();
7 BEGIN
8   familiafeliz.EXTEND (4);
9   familiafeliz(1) := 'Fernando';
10  familiafeliz(2) := 'Andres';
11  familiafeliz(3) := 'Gustavo';
12  familiafeliz(4) := 'Vilma';
13
14  hijos.EXTEND;
15  hijos (hijos.LAST) := 'Andres';
16  hijos.EXTEND;
17  hijos (hijos.LAST) := 'Gustavo';
18
19  padres := familiafeliz MULTISET EXCEPT hijos;
20
21  FOR l_row IN 1 .. padres.COUNT
22  LOOP
23    DBMS_OUTPUT.put_line (padres (l_row));
24  END LOOP;
25 END;
```

La siguiente tabla proporciona una explicación del código:

Líneas	Explicación
2	Se declara un nuevo tipo de tabla anidada. Cada elemento de una colección declarada con este tipo es una cadena cuya longitud máxima es 100 caracteres.
4 – 6	Se declaran tres tablas anidadas (familiafeliz, hijos y padres) en función del nuevo tipo de colección. Tener en cuenta que también se asigna un valor predeterminado a cada variable llamando a una función constructora que tiene el mismo nombre que el tipo.
8	Se hace espacio en la colección de familiafeliz para cuatro elementos llamando al método EXTEND
9 – 12	Se asignan los nombres de los miembros de la familia inmediata (el esposo: Fernando, los dos hijos: Andres y Gustavo, y la madre). Tener en cuenta el uso de la sintaxis típica de arreglos de una sola dimensión para identificar un elemento en el arreglo: nombre_arreglo (valor_índice).

14 – 17	Luego se rellena la tabla anidada de hijos sólo con los nombres de los hijos. En lugar de hacer una extensión "masiva" como en la línea 8, se extiende un valor de índice a la vez. Luego se asigna el nombre al valor de índice recién agregado llamando al método LAST, que devuelve el valor de índice definido más alto en la colección. A menos que se sepa cuántos elementos necesita de antemano, este enfoque de extender una fila y luego asignar un valor al nuevo valor de índice más alto es el camino a seguir.
19	Los dos hijos son adultos y se han mudado de la casa ancestral. Entonces, quiénes queda en ese lugar son el resto de la familiafeliz, por lo que a la familiafeliz se le restan (con el operador MULTiset EXCEPT) los hijos. Se asigna el resultado de esta operación de configuración a la colección principal. Deberían ser sólo Fernando y Vilma.
21 - 24	El resultado de una operación MULTiset siempre está vacío o densamente lleno y comienza con el valor de índice 1. Así que recorreré todos los elementos de la colección, desde 1 hasta COUNT (la cantidad de elementos definidos en la colección) y muestra el elemento encontrado en cada valor de índice.

Cuando se ejecuta el bloque anterior se ve el siguiente resultado:

```
Fernando
Vilma
```

Veamos cada una de las etapas que debemos seguir al utilizar las colecciones:

Declarar Tipos de Colección y Variables

Antes de que se pueda declarar y usar una variable de colección, debe definirse el tipo en el que se basa. Oracle Database predefine varios tipos de colección en los paquetes suministrados, como DBMS_SQL y DBMS_UTILITY. Sin embargo, en la mayoría de los casos, declararemos nuestros propios tipos de colección específicos de la aplicación. Aquí hay ejemplos de cómo declarar cada uno de los diferentes tipos de colecciones:

- Declare una matriz asociativa de números, indexada por entero

```
TYPE numeros_ta IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
```

- Declare una matriz asociativa de números, indexada por cadena:

```
TYPE numeros_ta IS TABLE OF NUMBER INDEX BY VARCHAR2(100);
```

- Declarar una tabla anidada de números:

```
TYPE numeros_tn IS TABLE OF NUMBER;
```

- Declarar una variedad de números:

```
TYPE numeros_va IS VARRAY(10) OF NUMBER;
```

Nota: se utilizaron los sufijos _ta, _tn y _va para matriz asociativa, tabla anidada y varray,.

Es frecuente preguntarse por qué la sintaxis para definir un tipo de colección no usa la palabra colección. La respuesta es que la sintaxis IS TABLE OF se introdujo por primera vez en Oracle7 Server,

cuando sólo había un tipo de colección, la tabla PL/SQL. A partir de estos ejemplos, puede sacar las siguientes conclusiones sobre los tipos de colección:

- Si la instrucción TYPE contiene una cláusula INDEX BY, el tipo de colección es una matriz asociativa.
- Si la instrucción TYPE contiene la palabra clave VARRAY, el tipo de colección es un varray.
- Si la instrucción TYPE no contiene una cláusula INDEX BY o una palabra clave VARRAY, el tipo de colección es una tabla anidada.
- Sólo la matriz asociativa ofrece una opción de tipos de datos de indexación. Las tablas anidadas, así como los varrays, siempre se indexan por número entero.
- Cuando se define un tipo de varray, se debe especificar el número máximo de elementos que se pueden definir en una colección de ese tipo.

Una vez que se haya declarado un tipo de colección, se lo puede usar para declarar una variable de colección como lo haría con cualquier otro tipo de variable, por ejemplo:

```
DECLARE
  TYPE numeros_tn IS TABLE OF NUMBER;
  l_numeros numeros_tn;
```

Inicializar Colecciones

Cuando trabaja con tablas anidadas y varrays, debe inicializar la variable de colección antes de poder usarla. Esto se hace llamando a la función constructora para ese tipo. Oracle Database crea automáticamente esta función cuando declara el tipo. La función constructora construye una instancia del tipo asociado con la función. Puede llamar a esta función sin argumentos, o puede pasarle una o más expresiones del mismo tipo que los elementos de la colección, y se insertarán en su colección. Veamos un ejemplo de cómo inicializar una tabla anidada de números con tres elementos (1, 2 y 3):

```
DECLARE
  TYPE numeros_tn IS TABLE OF NUMBER;
  l_numeros numeros_tn;
BEGIN
  l_numeros:= numeros_tn (1, 2, 3);
END;
```

Cuando no se inicializa una colección, Oracle Database generará un error cuando se intente usar esa colección:

```
1 DECLARE
2 TYPE numbers_nt IS TABLE OF NUMBER;
3   l_numbers numbers_nt;
4 BEGIN
5   l_numbers.EXTEND;
6   l_numbers(1) := 1;
7 END;
8 /

ERROR at line 1:
ORA-06531: Reference to uninitialized collection
ORA-06512: at line 5
```


No es necesario inicializar una matriz asociativa antes de asignarle valores.

Los vectores y las tablas anidadas podemos inicializarlas en la misma declaración utilizando la función constructora:

```
1 DECLARE
3
4  TYPE vector IS VARRAY(50) OF VARCHAR2(1);
5  letra vector:=vector();
```

También podemos inicializar y darle los contenidos a cada elemento de la colección en la declaración de este VARRAY, a algunos o a todos:

```
1 DECLARE
3  TYPE vector IS VARRAY(50) OF VARCHAR2(1);
4  numeros vector:=vector(11, 22, 43, 64, 25, 6);
```

Poblando Colecciones

Puede asignar valores a los elementos de una colección de varias formas:

- Se llama a una función constructora (para tablas anidadas y varrays).
- Se utiliza el operador de asignación (para elementos individuales y colecciones completas).
- Se pasa la colección a un subprograma como un parámetro OUT o IN OUT y luego se asigna el valor dentro del subprograma.
- Se utiliza una consulta BULK COLLECT.

La sección anterior incluía un ejemplo que usaba una función constructora. Los siguientes son ejemplos de los otros enfoques:

- Asignar un número a un sólo valor de índice. Tener en cuenta que con una matriz asociativa, no es necesario usar EXTEND o comenzar con el valor de índice 1

```
DECLARE
  TYPE numbers_aat IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l_numbers numbers_aat;
BEGIN
  l_numbers (100) := 12345;
END;
```

- Asignar una colección a otra. Siempre que ambas colecciones se declaren con el mismo tipo, puede realizar asignaciones a nivel de colección.

```
DECLARE
  TYPE numbers_aat IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l_numbers1 numbers_aat;
  l_numbers2 numbers_aat;
BEGIN
  l_numbers1 (100) := 12345;
  l_numbers2 := l_numbers1;
END;
```

- Pasar una colección como un argumento IN OUT en un procedimiento y eliminar todos los elementos de esa colección.

```
DECLARE
  TYPE numbers_aat IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l_numbers numbers_aat;
  PROCEDURE empty_collection (numbers_io IN OUT numbers_aat)
  IS
  BEGIN
    numbers_io.delete;
  END;
BEGIN
  l_numbers (100) := 123;
  empty_collection (l_numbers);
END;
```

- Completar una colección directamente desde una consulta con BULK COLLECT (este tema se cubrirá con más detalle al final de este apunte).

BULK COLLECT en PL/SQL es una cláusula que permite extraer varias filas de una consulta SQL y añadirlas a una colección PL/SQL en una sola operación. Esto mejora el rendimiento al minimizar la interacción entre el motor SQL y PL/SQL.

Con BULK COLLECT, se recuperan las filas de una consulta y se almacenan en una colección PL/SQL (como una matriz o un tipo de tabla). En lugar de procesar cada fila individualmente en un bucle, como se haría con FETCH, BULK COLLECT recopila todas las filas en una sola operación.

Veamos algunos ejemplos del uso de BULK COLLECT. Primero usemos un vector, aunque este no sería el mejor uso ya que al desconocer cuántos registros se tienen en la Tabla hay que reservar una cantidad superior a lo posible de usar, en caso de que en un VARRAY se declare con menos posiciones que la cantidad de registros dará error y cortará con la ejecución:

```
DECLARE
  TYPE regclie_ta IS VARRAY(50) OF clientes%rowtype;
  cliente regclie_ta;
BEGIN
  SELECT * BULK COLLECT INTO cliente FROM clientes;
  FOR I IN cliente.FIRST..cliente.LAST LOOP
    dbms_output.put_line(cliente(I).nombres || ' - ' || cliente(I).distrito );
  END LOOP;
END;
```

Veamos ahora el uso de BULK COLLECT en una tabla anidada:

```
DECLARE
  TYPE registro_ta IS TABLE OF criminales%rowtype;
  criminal registro_ta;
BEGIN
  SELECT * BULK COLLECT INTO criminal FROM criminales ORDER BY nombre;
  FOR I IN criminal.FIRST..criminal.LAST LOOP
    dbms_output.put_line(to_char(criminal(I).ID) || ' - ' || criminal(I).nombre);
  END LOOP;
END;
```

Veamos ahora el uso de BULK COLLECT en un arreglo asociativo:

```
DECLARE
  TYPE registro_ta IS TABLE OF criminales%rowtype INDEX BY PLS_INTEGER;
  criminal registro_ta;
BEGIN
  SELECT * BULK COLLECT INTO criminal FROM criminales ORDER BY id;
  FOR I IN criminal.FIRST..criminal.LAST LOOP
    dbms_output.put_line(to_char(criminal(I).ID) || ' - ' || criminal(I).nombre);
  END LOOP;
END;
```

Beneficios de BULK COLLECT

- **Mejora el rendimiento:** Reduce la cantidad de cambios de contexto entre el motor SQL y el motor PL/SQL, lo que puede aumentar significativamente la velocidad del procesamiento de grandes conjuntos de datos.
- **Simplifica el código:** Facilita la manipulación de múltiples filas de una consulta en una sola operación.
- **Optimiza el uso de recursos:** Permite procesar grandes cantidades de datos de forma eficiente, reduciendo la necesidad de utilizar recursos del sistema.

Consideraciones de BULK COLLECT

- **Limitaciones de memoria:** Si se intenta recuperar demasiadas filas en una sola operación, podría generar errores de memoria. Se recomienda utilizar LIMIT con BULK COLLECT para controlar la cantidad de filas que se recuperan en cada operación.

Iterando a Través de Colecciones

Una operación de colección muy común es iterar a través de todos los elementos de una colección.

Las razones para realizar un "análisis completo de la colección" incluyen mostrar información en la colección, ejecutar una declaración de lenguaje de manipulación de datos (DML) con datos en el elemento y buscar datos específicos.

El tipo de código que escribe para iterar a través de una colección está determinado por el tipo de colección con la que está trabajando y cómo se completó. En general, se elegirá entre un bucle **FOR** numérico y un bucle **WHILE**.

Utilice un bucle **FOR** numérico cuando:

- La colección está densamente llena (se define cada valor de índice entre el menor y el mayor)
- Se desea escanear toda la colección y no terminarlo si se cumple alguna condición.

Por el contrario, utilice un ciclo **WHILE** cuando

- Su colección puede ser escasa.
- Puede terminar el bucle antes de haber iterado a través de todos los elementos de la colección.

No se debe usar un bucle **FOR** numérico con colecciones densas para evitar una excepción **NO_DATA_FOUND**.

Pero, Oracle también generará la excepción **NO_DATA_FOUND** si intenta leer un elemento en una colección en un valor de índice indefinido.

El siguiente bloque, por ejemplo, genera una excepción **NO_DATA_FOUND**:

```
DECLARE
  TYPE numeros_aa IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l_numeros numeros_aa;
BEGIN
  DBMS_OUTPUT.PUT_LINE (l_numeros (100));
END;
```

Sin embargo, cuando se sabe con certeza que su colección está, y siempre estará, densamente llena, el ciclo FOR ofrece el código más simple para hacer el trabajo.

El Paquete DBMS_UTILITY

El paquete DBMS_UTILITY en Oracle proporciona una serie de subprogramas de utilidad para diversas tareas, como la gestión de errores, la generación de números aleatorios, y la manipulación de datos. Es una herramienta esencial para el desarrollo de aplicaciones en PL/SQL.

Subprogramas clave de DBMS_UTILITY:

- **FORMAT_ERROR_BACKTRACE:** Permite obtener un rastreo detallado de errores, incluyendo la línea de código donde se generó el error.
- **GET_MAX_NUMBER:** Devuelve el número máximo que puede ser almacenado en un tipo de datos determinado.
- **GET_DB_VERSION:** Obtiene la versión del sistema de base de datos Oracle.
- **GET_COMPATIBLE_VERSION:** Obtiene el valor de compatibilidad de la base de datos.
- **GET_HOST_NAME:** Devuelve el nombre de la máquina donde se ejecuta la base de datos.
- **INVALIDATE:** Permite invalidar objetos de la base de datos.
- **GET_PID:** Devuelve el ID del proceso de la sesión actual.
- **SUBSTR:** Permite extraer una subcadena de una cadena de caracteres.
- **TO_CLOB:** Convierte un valor VARCHAR2 en un valor CLOB.
- **TO_CHAR:** Convierte un valor en una cadena de caracteres.
- **TO_NUMBER:** Convierte una cadena de caracteres en un número.

Usos comunes de DBMS_UTILITY:

- **Gestión de errores:** FORMAT_ERROR_BACKTRACE es invaluable para depurar aplicaciones, ya que proporciona información detallada sobre el origen de los errores.
- **Generación de números aleatorios:** Aunque hay otros paquetes como DBMS_RANDOM, DBMS_UTILITY también puede ser utilizado para esta tarea.
- **Manipulación de datos:** Funciones como SUBSTR, TO_CLOB, TO_CHAR y TO_NUMBER son útiles para la transformación de datos.
- **Información del sistema:** GET_DB_VERSION, GET_HOST_NAME y otros subprogramas permiten obtener información del entorno de la base de datos.

En resumen, el paquete DBMS_UTILITY ofrece una gama de funciones útiles para la administración y el desarrollo de aplicaciones en Oracle, especialmente para la gestión de errores, la generación de números aleatorios, la manipulación de datos, análisis de objetos, compilación, resolución de nombres entre otros.

DBMS_UTILITY.maxname_array

El **maxname_array** del paquete DBMS_UTILITY no es un subprograma en sí mismo, sino el nombre de un tipo de tabla PL/SQL específico que se utiliza para almacenar una matriz PL/SQL de valores VARCHAR2. Esta matriz es utilizada por ciertos procedimientos DBMS_UTILITY para almacenar los resultados de la resolución de nombres u otras operaciones.

Es un tipo predefinido para almacenar una colección de valores VARCHAR2, diseñado específicamente para almacenar nombres. Lo utilizan algunos procedimientos para gestionar una lista de nombres separados por comas y convertirlos en una tabla PL/SQL de cadenas.

En el contexto de los nombres de objetos, ayuda a resolverlos y almacenarlos, incluyendo potencialmente información del propietario o del esquema.

No es un procedimiento ni una función que se ejecute directamente, sino una definición de tipo que se utiliza en otros subprogramas de DBMS_UTILITY.

En esencia, DBMS_UTILITY.maxname_array es una herramienta utilizada internamente por el paquete DBMS_UTILITY para gestionar operaciones relacionadas con nombres, especialmente al trabajar con listas de nombres en la base de datos.

Permite el procesamiento y almacenamiento eficiente de estos nombres en una tabla PL/SQL.

Tener en cuenta que la lista debe ser una lista no vacía delimitada por comas. Se rechaza cualquier lista que no sea una lista delimitada por comas.

Veamos un ejemplo del uso de paquete DBMS_UTILITY con la definición de nombres dada por maxname_array:

El siguiente procedimiento muestra todas las cadenas de una colección cuyo tipo se define en el paquete DBMS_UTILITY.

```
CREATE OR REPLACE PROCEDURE mostrar_contenidos
(nombres IN DBMS_UTILITY.maxname_array)
IS
BEGIN
    FOR indx IN nombres.FIRST .. nombres.LAST LOOP
        DBMS_OUTPUT.put_line (nombres (indx));
    END LOOP;
END;
```

El procedimiento anterior llama a dos métodos: FIRST y LAST. FIRST devuelve el valor de índice definido más bajo de la colección y LAST devuelve el valor de índice definido más alto de la colección. El siguiente bloque mostrará los nombres de tres artistas; tenga en cuenta que los valores de índice no necesitan comenzar en 1.

```
DECLARE
    l_nombres DBMS_UTILITY.maxname_array;
BEGIN
    l_nombres (100) := 'Picasso';
    l_nombres (101) := 'O''Keefe';
    l_nombres (102) := 'Dali';
    mostrar_contenido (l_nombres);
END;
```

Si una colección puede ser escasa o si se desea terminar el bucle condicionalmente, un bucle WHILE será la mejor opción.

El siguiente procedimiento muestra este enfoque.


```
CREATE OR REPLACE PROCEDURE mostrar_contenidos
(nombres IN DBMS_UTILITY.maxname_array)
IS
    l_index PLS_INTEGER := nombres.FIRST;
BEGIN
    WHILE (l_index IS NOT NULL) LOOP
        DBMS_OUTPUT.put_line (nombres(l_index));
        l_index := nombres.NEXT (l_index);
    END LOOP;
END;
```

En este último procedimiento, el iterador (l_index) se establece inicialmente en el valor de índice definido más bajo. Si la colección está vacía, tanto FIRST (primero) como LAST (último) devolverán NULL. El ciclo WHILE termina cuando l_index es NULL. Luego se muestra el nombre en el valor de índice actual y se llama al método NEXT para obtener el siguiente valor de índice definido más alto que l_index. Esta función devuelve NULL cuando no hay un valor de índice más alto.

Llamo a este procedimiento en el siguiente bloque, con una colección que no se llena secuencialmente. Mostrará los tres nombres sin generar NO_DATA_FOUND:

```
DECLARE
    l_nombres DBMS_UTILITY.maxname_array;
BEGIN
    l_nombres (-150) := 'Picasso';
    l_nombres (0) := 'O''Keefe';
    l_nombres (307) := 'Dali';
    mostrar_contenido(l_nombres); END;
```

También puedo escanear el contenido de una colección al revés, comenzando con LAST y usando el método PRIOR, como se muestra en el siguiente procedimiento.

```
CREATE OR REPLACE PROCEDURE mostrar_contenidos
(nombres IN DBMS_UTILITY.maxname_array)
IS
    l_index PLS_INTEGER := nombres.LAST;
BEGIN
    WHILE (l_index IS NOT NULL) LOOP
        DBMS_OUTPUT.put_line (nombres (l_index));
        l_index := nombres.PRIOR (l_index);
    END LOOP;
END;
```

Eliminar elementos de la colección

PL/SQL ofrece un método DELETE, que puede usar para eliminar todos, uno o algunos elementos de una colección. Aquí hay unos ejemplos:

- Eliminar todos los elementos de una colección; use el método DELETE sin ningún argumento. Esta forma de eliminar funciona con los tres tipos de colecciones.

```
l_nombres.DELETE;
```

- Eliminar el primer elemento de una colección; para eliminar un elemento, pase el valor del índice a DELETE. Esta forma de DELETE solo se puede usar con una matriz asociativa o una tabla anidada.

```
I_nombres.DELETE (I_nombres.FIRST);
```

- Elimina todos los elementos entre los valores de índice alto y bajo especificados. Esta forma de DELETE solo se puede usar con una matriz asociativa o una tabla anidada.

```
I_nombres.DELETE (100, 200);
```

- Si especifica un valor de índice indefinido, Oracle Database no generará un error.
- También puede usar el método TRIM con varrays y tablas anidadas para eliminar elementos del final de la colección. Puede recortar uno o varios elementos:

```
I_nombres.TRIM; I_nombres.TRIM (3);
```

Es imposible aprovechar al máximo PL/SQL sin usar colecciones. Pero, sin embargo, brevemente podemos decir que un paquete es un contenedor para múltiples funciones y procedimientos. Oracle amplía PL/SQL con muchos paquetes integrados o suministrados.

Selección de Tipo de Colección

La elección del tipo de colección PL/SQL más adecuado depende de las necesidades de procesamiento. Por ejemplo, si se necesita acceder a los elementos de la colección por índice, se puede utilizar una matriz. Si se necesita acceder a los elementos por un valor de llave, se puede utilizar un tipo de tabla asociado.

BULK COLLECT y FORALL

Hasta acá se presentaron las colecciones, importantes estructuras de datos que resultan muy útiles al implementar algoritmos que manipulan listas de datos de programa. Las colecciones también son elementos clave en algunas de las potentes funciones de optimización del rendimiento de PL/SQL. A continuación se abordarán las dos funciones más importantes de las colecciones: BULK COLLECT y FORALL.

En Oracle, "bulk data" (datos masivos) se refiere a la carga y manipulación eficientes de grandes volúmenes de datos, utilizando técnicas como BULK COLLECT para la selección y FORALL para la modificación (insert, update, delete) de datos en colecciones, mejorando significativamente el rendimiento.

- **BULK COLLECT:** es una cláusula en PL/SQL que permite recuperar múltiples filas de una consulta en una sola ejecución, almacenándolas en una colección (array). Esto mejora la velocidad de recuperación de datos en comparación con la recuperación fila. Son sentencias SELECT que recuperan múltiples filas con una sola búsqueda, lo que mejora la velocidad de recuperación de datos.
- **FORALL:** FORALL es una instrucción en PL/SQL que permite ejecutar operaciones (INSERT, UPDATE, DELETE) en múltiples filas de una colección de manera eficiente, muy rápidamente. Permite realizar cambios en grandes cantidades de datos de forma rápida.

¿Pero cuál es el verdadero impacto de estas funciones? Los resultados reales variarán según la versión de Oracle Database que se esté ejecutando y las características específicas de la lógica de la aplicación.

Dado que PL/SQL está tan estrechamente integrado con el lenguaje SQL, lo primero que surge en mente es si es necesario tener funciones especiales para mejorar el rendimiento de las sentencias SQL dentro de PL/SQL. Pero la explicación radica en cómo se comunican entre sí los motores de ejecución de PL/SQL y SQL: se comunican mediante un cambio de contexto.

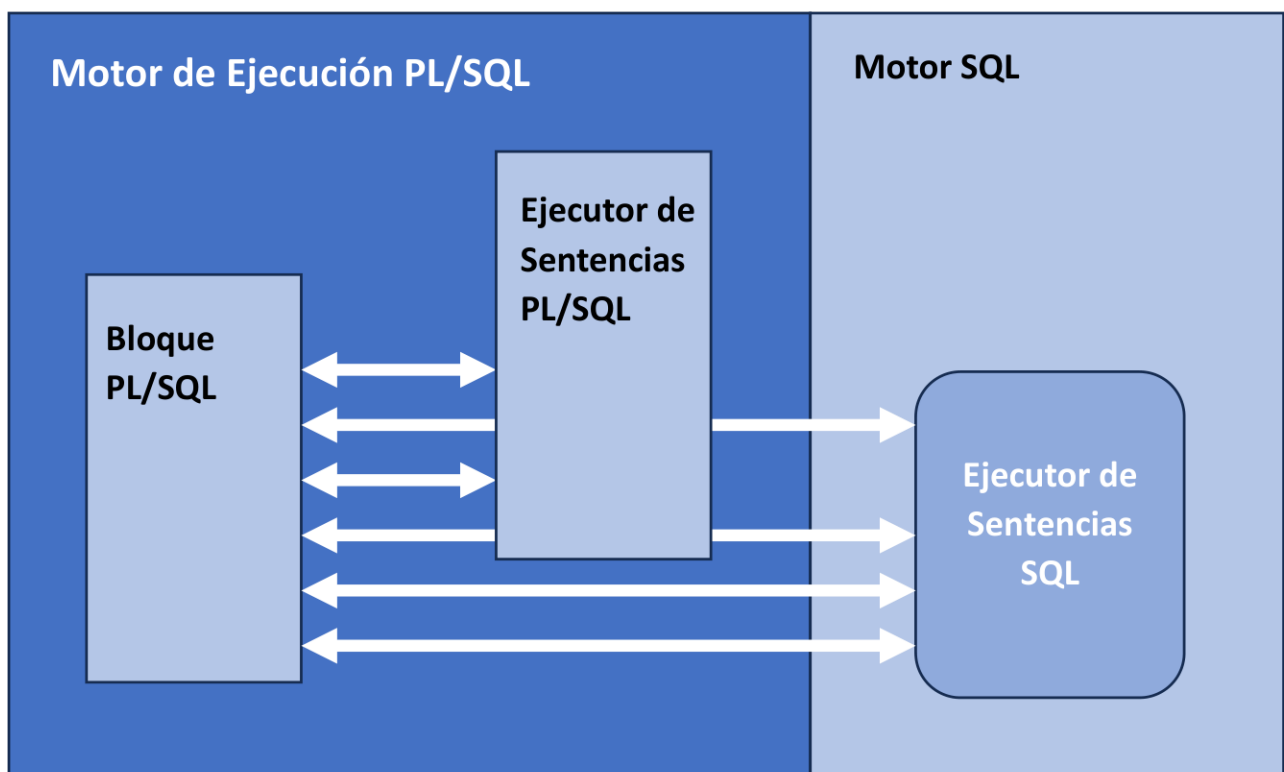
Ejemplo de uso

```
-- BULK COLLECT para seleccionar datos
SELECT * INTO my_collection FROM employees;

-- FORALL para insertar datos
FORALL i IN 1..my_collection.COUNT
INSERT INTO new_table VALUES (my_collection(i).id, my_collection(i).name);
```

Cambios de contexto y rendimiento

Casi todos los programas PL/SQL incluyen sentencias PL/SQL y SQL. Las sentencias PL/SQL las ejecuta el ejecutor de sentencias PL/SQL; las sentencias SQL las ejecuta el ejecutor de sentencias SQL. Cuando el motor de ejecución de PL/SQL encuentra una sentencia SQL, se detiene y la pasa al motor SQL. El motor SQL ejecuta la sentencia SQL y devuelve la información al motor PL/SQL. Esta transferencia de control se denomina cambio de contexto y cada uno de estos cambios genera una sobrecarga que reduce el rendimiento general de los programas.



Veamos un ejemplo concreto para explorar los cambios de contexto con más detalle e identificar la razón por la que FORALL y BULK COLLECT pueden tener un impacto tan drástico en el rendimiento.

Ejemplo: Supongamos que se me pidió que escribiera un procedimiento que aceptara un ID de departamento y un aumento porcentual de salario, y que otorgara a todos los empleados de ese departamento un aumento del porcentaje especificado. Aprovechando el elegante bucle FOR de

cursor de PL/SQL y la posibilidad de invocar sentencias SQL de forma nativa en PL/SQL, creé el siguiente código

```
CREATE OR REPLACE PROCEDURE aumento_sal
  (dpto_id IN empleados.departamento_id%TYPE, aumento_pct IN NUMBER)
IS
BEGIN
  FOR emplereg IN
    (SELECT id FROM empleados WHERE departamento_id = aumento_sal.departamento_id) LOOP
    UPDATE empleados e SET e.salario = e.salario + e.salario * aumento_sal.aumento_pct
      WHERE emp. id = emplereg.id;
    END LOOP;
END;
```

Supongamos que hay 100 empleados en el departamento 15. Cuando ejecuto este bloque,

```
BEGIN
  aumento_sal (15, .10);
END;
```

El motor PL/SQL cambiará al motor SQL 100 veces, una por cada fila que se actualice. Podríamos referirnos a este cambio fila por fila como "procesamiento lento a lento", y definitivamente es algo que debe evitarse. Podemos usar las funciones de procesamiento masivo de PL/SQL para evitar el "procesamiento lento a lento". Sin embargo, primero debe verificarse si es posible evitar el cambio de contexto entre PL/SQL y SQL realizando la mayor parte del trabajo posible dentro de SQL.

Revisemos nuevamente el procedimiento **aumento_sal**. La sentencia SELECT identifica a todos los empleados de un departamento. La sentencia UPDATE se ejecuta para cada uno de esos empleados, aplicando el mismo porcentaje de aumento a todos. En un escenario tan simple, no se necesita un bucle FOR de cursor. Puedo simplificar este procedimiento simplemente con el código que a continuación se muestra:

```
CREATE OR REPLACE PROCEDURE aumento_sal
  (dpto_id IN empleados.departamento_id%TYPE, aumento_pct IN NUMBER)
IS
BEGIN
  UPDATE empleados e SET e.salario = e.salario + e.salario * aumento_sal.aumento_pct
    WHERE emp. id = emplereg.id;
END LOOP;
END;
```

En realidad bastó con un cambio de contexto para ejecutar una sentencia UPDATE. Todo el trabajo ahora se realizará en el motor SQL.

Por supuesto, en la mayoría de los escenarios reales, la vida —y el código— no son tan sencillos. A menudo necesitamos realizar otros pasos antes de ejecutar nuestras sentencias de lenguaje de manipulación de datos (DML).

Supongamos que, por ejemplo, en el caso del procedimiento **aumento_salario**, necesito comprobar si los empleados cumplen los requisitos para el aumento de salario y, si no los cumplen, enviar una notificación por correo electrónico.

```
CREATE OR REPLACE PROCEDURE aumento_sal
  (dpto_id IN empleados.departamento_id%TYPE, aumento_pct IN NUMBER)
IS
  l_elegible BOOLEAN;
BEGIN
  FOR emplereg IN
    (SELECT id FROM empleados WHERE departamento_id = aumento_sal.departamento_id) LOOP
    controlar_elegible (emplereg.id, aumento_pct, l_elegible);
    IF l_elegible THEN
      UPDATE empleados e SET e.salario = e.salario + e.salario * aumento_sal.aumento_pct
        WHERE emp.id = emplereg.id;
    END IF;
  END LOOP;
END;
```

Ya no se puede hacerlo todo en SQL, así que debemos resignarnos al "procesamiento lento a lento" o utilizar BULK COLLECT y FORALL en PL/SQL.

Procesamiento Masivo de Datos en PL/SQL

Las funciones de procesamiento masivo de PL/SQL están diseñadas específicamente para reducir el número de cambios de contexto necesarios para la comunicación entre el motor PL/SQL y el motor SQL.

Utilizar la cláusula BULK COLLECT para extraer varias filas en una o más colecciones con un solo cambio de contexto.

Utilizar la sentencia FORALL cuando se necesite ejecutar la misma sentencia DML repetidamente para diferentes valores de variables de enlace. La sentencia UPDATE del procedimiento increase_salary se adapta a este escenario; lo único que cambia con cada nueva ejecución de la sentencia es el ID del empleado.